



Pandas

Pandas (Panel Data) is a Python library. Pandas is used to analyze data. It is built on top of Numpy (Numerical Python). It helps to read data from various sources such as CSV, JSON, excel files into pandas object called Series/DataFrames. It is capable of reading heterogeneous data. It has flexible methods to clean and analyze the data.

Pandas Basics

1. Importing libraries
2. Loading dataset
3. Initial inspection
4. Row retrieval
5. Column retrieval
6. Adding new rows
7. Adding new columns
8. Subsetting
9. Filtering
10. Sorting
11. Basic Aggregate operations
12. Saving a csv file

Importing Libraries

```
In [1]: import pandas as pd  
print("Pandas imported successfully")
```

Pandas imported successfully

Loading the Dataset

```
In [2]: df = pd.read_csv("Salaries.csv")  
print("Dataset loaded successfully")
```

Dataset loaded successfully

Initial inspection

```
In [3]: # Display first 5 rows  
df.head()
```

```
Out[3]:
```

	Unnamed: 0	rank	discipline	yrs.since.phd	yrs.service	sex	salary
0	1	Prof	B	19	18	Male	139750
1	2	Prof	B	20	16	Male	173200
2	3	AsstProf	B	4	3	Male	79750
3	4	Prof	B	45	39	Male	115000
4	5	Prof	B	40	41	Male	141500

```
In [4]: # Display last 5 rows
df.tail()
```

```
Out[4]:
```

	Unnamed: 0	rank	discipline	yrs.since.phd	yrs.service	sex	salary
392	393	Prof	A	33	30	Male	103106
393	394	Prof	A	31	19	Male	150564
394	395	Prof	A	42	25	Male	101738
395	396	Prof	A	25	15	Male	95329
396	397	AsstProf	A	8	4	Male	81035

```
In [5]: # display 10 rows
df.head(10)
```

```
Out[5]:
```

	Unnamed: 0	rank	discipline	yrs.since.phd	yrs.service	sex	salary
0	1	Prof	B	19	18	Male	139750
1	2	Prof	B	20	16	Male	173200
2	3	AsstProf	B	4	3	Male	79750
3	4	Prof	B	45	39	Male	115000
4	5	Prof	B	40	41	Male	141500
5	6	AssocProf	B	6	6	Male	97000
6	7	Prof	B	30	23	Male	175000
7	8	Prof	B	45	45	Male	147765
8	9	Prof	B	21	20	Male	119250
9	10	Prof	B	18	18	Female	129000

```
In [6]: # Check the shape of the dataset
df.shape
```

Out[6]: (397, 7)

```
In [7]: # Check column names
df.columns
```

Out[7]: Index(['Unnamed: 0', 'rank', 'discipline', 'yrs.since.phd', 'yrs.service',
 'sex', 'salary'],
 dtype='object')

```
In [8]: # Check data types
df.dtypes
```

Out[8]: Unnamed: 0 int64
rank object
discipline object
yrs.since.phd int64
yrs.service int64
sex object
salary int64
dtype: object

```
In [9]: # General information
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 397 entries, 0 to 396
Data columns (total 7 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Unnamed: 0      397 non-null   int64
1   rank            397 non-null   object
2   discipline      397 non-null   object
3   yrs.since.phd   397 non-null   int64
4   yrs.service     397 non-null   int64
5   sex            397 non-null   object
6   salary         397 non-null   int64
dtypes: int64(4), object(3)
memory usage: 21.8+ KB
```

```
In [10]: # Statistical summary of numerical columns
df.describe()
```

```
Out[10]:
```

	Unnamed: 0	yrs.since.phd	yrs.service	salary
count	397.000000	397.000000	397.000000	397.000000
mean	199.000000	22.314861	17.614610	113706.458438
std	114.748275	12.887003	13.006024	30289.038695
min	1.000000	1.000000	0.000000	57800.000000
25%	100.000000	12.000000	7.000000	91000.000000
50%	199.000000	21.000000	16.000000	107300.000000
75%	298.000000	32.000000	27.000000	134185.000000
max	397.000000	56.000000	60.000000	231545.000000

```
In [11]: # Statistical summary of all columns
df.describe(include="all")
```

```
Out[11]:
```

	Unnamed: 0	rank	discipline	yrs.since.phd	yrs.service	sex	sa
count	397.000000	397	397	397.000000	397.000000	397	397.000000
unique	NaN	3	2	NaN	NaN	2	NaN
top	NaN	Prof	B	NaN	NaN	Male	NaN
freq	NaN	266	216	NaN	NaN	358	NaN
mean	199.000000	NaN	NaN	22.314861	17.614610	NaN	113706.458438
std	114.748275	NaN	NaN	12.887003	13.006024	NaN	30289.038695
min	1.000000	NaN	NaN	1.000000	0.000000	NaN	57800.000000
25%	100.000000	NaN	NaN	12.000000	7.000000	NaN	91000.000000
50%	199.000000	NaN	NaN	21.000000	16.000000	NaN	107300.000000
75%	298.000000	NaN	NaN	32.000000	27.000000	NaN	134185.000000
max	397.000000	NaN	NaN	56.000000	60.000000	NaN	231545.000000

Row Retrieval

```
In [12]: # Retrieve first row using iloc
df.iloc[0]
```

```
Out[12]: Unnamed: 0      1
rank      Prof
discipline B
yrs.since.phd 19
yrs.service 18
sex      Male
salary    139750
Name: 0, dtype: object
```

```
In [13]: # Retrieve third row
df.iloc[2]
```

```
Out[13]: Unnamed: 0      3
rank      AsstProf
discipline B
yrs.since.phd 4
yrs.service 3
sex      Male
salary    79750
Name: 2, dtype: object
```

```
In [14]: # Retrieve multiple rows: example rows 11 to 15
df.iloc[11:16]
```

```
Out[14]:
```

	Unnamed: 0	rank	discipline	yrs.since.phd	yrs.service	sex	salary
11	12	AsstProf	B	7	2	Male	79800
12	13	AsstProf	B	1	1	Male	77700
13	14	AsstProf	B	2	0	Male	78000
14	15	Prof	B	20	18	Male	104800
15	16	Prof	B	12	3	Male	117150

```
In [15]: # Retrieve specific rows: row 2, 4, and 6
df.iloc[[2, 4, 6]]
```

```
Out[15]:
```

	Unnamed: 0	rank	discipline	yrs.since.phd	yrs.service	sex	salary
2	3	AsstProf	B	4	3	Male	79750
4	5	Prof	B	40	41	Male	141500
6	7	Prof	B	30	23	Male	175000

Column Retrieval

```
In [16]: # Retrieve one column by column heading
df["salary"]
```

```
Out[16]: 0      139750
         1      173200
         2       79750
         3     115000
         4     141500
         ...
        392    103106
        393    150564
        394    101738
        395     95329
        396     81035
        Name: salary, Length: 397, dtype: int64
```

```
In [17]: # Retrieve multiple columns by column heading
         df[["rank", "salary"]]
```

```
Out[17]:
```

	rank	salary
0	Prof	139750
1	Prof	173200
2	AsstProf	79750
3	Prof	115000
4	Prof	141500
...	...	...
392	Prof	103106
393	Prof	150564
394	Prof	101738
395	Prof	95329
396	AsstProf	81035

397 rows × 2 columns

```
In [18]: # Retrieve columns by column position
         df.iloc[:,1:4]
```

Out[18]:

	rank	discipline	yrs.since.phd
0	Prof	B	19
1	Prof	B	20
2	AsstProf	B	4
3	Prof	B	45
4	Prof	B	40
...	...	...	...
392	Prof	A	33
393	Prof	A	31
394	Prof	A	42
395	Prof	A	25
396	AsstProf	A	8

397 rows × 3 columns

```
In [19]: # Retrieve specific columns by column position
df.iloc[:,[1,4,5]]
```

Out[19]:

	rank	yrs.service	sex
0	Prof	18	Male
1	Prof	16	Male
2	AsstProf	3	Male
3	Prof	39	Male
4	Prof	41	Male
...	...	...	...
392	Prof	30	Male
393	Prof	19	Male
394	Prof	25	Male
395	Prof	15	Male
396	AsstProf	4	Male

397 rows × 3 columns

Adding a New Row

```
In [20]: new_row = {  
    "rank": "AsstProf",  
    "discipline": "A",  
    "yrs.since.phd": 8,  
    "yrs.service": 5,  
    "sex": "Female",  
    "salary": 85000,  
    }
```

```
In [21]: # Add new row using loc  
df.loc[len(df)] = new_row
```

```
In [22]: df.tail()
```

```
Out[22]:
```

	Unnamed: 0	rank	discipline	yrs.since.phd	yrs.service	sex	salary
393	394.0	Prof	A	31	19	Male	150564
394	395.0	Prof	A	42	25	Male	101738
395	396.0	Prof	A	25	15	Male	95329
396	397.0	AsstProf	A	8	4	Male	81035
397	NaN	AsstProf	A	8	5	Female	85000

```
In [23]: ### Adding a New Column  
df["salary_in_lakhs"] = df["salary"] / 100000
```

```
In [24]: df.head()
```

```
Out[24]:
```

	Unnamed: 0	rank	discipline	yrs.since.phd	yrs.service	sex	salary	salary_in_lakhs
0	1.0	Prof	B	19	18	Male	139750	1.3975
1	2.0	Prof	B	20	16	Male	173200	1.732
2	3.0	AsstProf	B	4	3	Male	79750	0.7975
3	4.0	Prof	B	45	39	Male	115000	1.15
4	5.0	Prof	B	40	41	Male	141500	1.415

Subsetting

```
In [25]: # Select first 10 rows and first 4 columns  
s1=df.iloc[0:10, 0:4]  
s1.head()
```


Out[25]:

	Unnamed: 0	rank	discipline	yrs.since.phd
0	1.0	Prof	B	19
1	2.0	Prof	B	20
2	3.0	AsstProf	B	4
3	4.0	Prof	B	45
4	5.0	Prof	B	40

In [26]:

```
# Select only selected columns
s2=df[["rank", "sex", "salary"]]
s2.head()
```

Out[26]:

	rank	sex	salary
0	Prof	Male	139750
1	Prof	Male	173200
2	AsstProf	Male	79750
3	Prof	Male	115000
4	Prof	Male	141500

In [27]:

```
# Select rows 5 to 10 with selected columns
df.loc[5:10, ["rank", "yrs.service", "salary"]]
```

Out[27]:

	rank	yrs.service	salary
5	AssocProf	6	97000
6	Prof	23	175000
7	Prof	45	147765
8	Prof	20	119250
9	Prof	18	129000
10	AssocProf	8	119800

Filtering

In [28]:

```
# Filter employees with salary greater than 120000 (Handling numerical column)
high_salary = df[df["salary"] > 120000]
high_salary
```

Out[28]:

	Unnamed: 0	rank	discipline	yrs.since.phd	yrs.service	sex	salary	salary
0	1.0	Prof	B	19	18	Male	139750	
1	2.0	Prof	B	20	16	Male	173200	
4	5.0	Prof	B	40	41	Male	141500	
6	7.0	Prof	B	30	23	Male	175000	
7	8.0	Prof	B	45	45	Male	147765	
...	...	...	...	...	...	...	...	...
386	387.0	Prof	A	29	27	Male	139219	
389	390.0	Prof	A	33	18	Male	186023	
390	391.0	Prof	A	40	19	Male	166605	
391	392.0	Prof	A	30	19	Male	151292	
393	394.0	Prof	A	31	19	Male	150564	

142 rows × 8 columns

In [29]: *# Filter only Professors (Handling categorical column)*
 prof=df[df["rank"] == "Prof"]
 prof

Out[29]:

	Unnamed: 0	rank	discipline	yrs.since.phd	yrs.service	sex	salary	salary
0	1.0	Prof	B	19	18	Male	139750	
1	2.0	Prof	B	20	16	Male	173200	
3	4.0	Prof	B	45	39	Male	115000	
4	5.0	Prof	B	40	41	Male	141500	
6	7.0	Prof	B	30	23	Male	175000	
...	...	...	...	...	...	...	...	...
391	392.0	Prof	A	30	19	Male	151292	
392	393.0	Prof	A	33	30	Male	103106	
393	394.0	Prof	A	31	19	Male	150564	
394	395.0	Prof	A	42	25	Male	101738	
395	396.0	Prof	A	25	15	Male	95329	

266 rows × 8 columns

In [30]: *# Multiple conditions*

```
filtered_data = df[(df["rank"] == "Prof") & (df["salary"] > 130000)]
filtered_data
```

```
Out[30]:
```

	Unnamed: 0	rank	discipline	yrs.since.phd	yrs.service	sex	salary	salary
0	1.0	Prof	B	19	18	Male	139750	
1	2.0	Prof	B	20	16	Male	173200	
4	5.0	Prof	B	40	41	Male	141500	
6	7.0	Prof	B	30	23	Male	175000	
7	8.0	Prof	B	45	45	Male	147765	
...	...	...	...	...	...	...	...	...
386	387.0	Prof	A	29	27	Male	139219	
389	390.0	Prof	A	33	18	Male	186023	
390	391.0	Prof	A	40	19	Male	166605	
391	392.0	Prof	A	30	19	Male	151292	
393	394.0	Prof	A	31	19	Male	150564	

110 rows × 8 columns

Sorting

```
In [31]: #Sort ascending
df.sort_values(by="salary")
```

Out[31]:

	Unnamed: 0	rank	discipline	yrs.since.phd	yrs.service	sex	salary
282	283.0	Prof	A	51	51	Male	57800
123	124.0	AssocProf	A	25	22	Female	62884
237	238.0	AsstProf	A	7	6	Female	63100
226	227.0	AsstProf	A	3	1	Male	63900
317	318.0	Prof	B	46	45	Male	67559
...	...	...	...	...	...	...	...
77	78.0	Prof	B	26	19	Male	193000
271	272.0	Prof	A	42	18	Male	194800
249	250.0	Prof	A	29	7	Male	204000
364	365.0	Prof	A	43	43	Male	205500
43	44.0	Prof	B	38	38	Male	231545

398 rows × 8 columns

In [32]: `# Sort Descending`
`df.sort_values(by="salary", ascending=False)`

Out[32]:

	Unnamed: 0	rank	discipline	yrs.since.phd	yrs.service	sex	salary
43	44.0	Prof	B	38	38	Male	231545
364	365.0	Prof	A	43	43	Male	205500
249	250.0	Prof	A	29	7	Male	204000
271	272.0	Prof	A	42	18	Male	194800
77	78.0	Prof	B	26	19	Male	193000
...	...	...	...	...	...	...	...
317	318.0	Prof	B	46	45	Male	67559
226	227.0	AsstProf	A	3	1	Male	63900
237	238.0	AsstProf	A	7	6	Female	63100
123	124.0	AssocProf	A	25	22	Female	62884
282	283.0	Prof	A	51	51	Male	57800

398 rows × 8 columns

Basic Aggregate Functions

```
In [33]: # mean  
df["salary"].mean()
```

```
Out[33]: np.float64(113634.33165829146)
```

```
In [34]: # round  
df["salary"].mean().round(2)
```

```
Out[34]: np.float64(113634.33)
```

```
In [35]: #max  
df["salary"].max()
```

```
Out[35]: 231545
```

```
In [36]: # min  
df["salary"].min()
```

```
Out[36]: 57800
```

```
In [37]: # Unique  
df["sex"].unique()
```

```
Out[37]: array(['Male', 'Female'], dtype=object)
```

```
In [38]: # Frequency  
df["sex"].value_counts()
```

```
Out[38]: sex  
Male      358  
Female     40  
Name: count, dtype: int64
```

Saving the Dataset

```
In [39]: df.to_csv("Salaries_updated.csv", index=False)
```

Done!!!